

UNIVERSIDAD INVENIO

TAREA II DE INVESTIGACIÓN GRUPAL

Patrones Arquitecturales de Software Más Comunes

GRUPO 1 – ARQUITECTURA MONOLÍTICA

INTEGRANTES:

Alejandro Zamora

Alejandro Luna

César Ubau

Saúl Ramírez

Profesor: Darin Mauricio Gamboa

Curso: Ingeniería de Software I

Índice de Contenidos

Índice de Figuras y Tablas	4
1. Introducción	5
2. Definición Formal de la Arquitectura Monolítica	6
3. Historia y Evolución del Patrón	8
3.1 Orígenes (Décadas de 1950–1980)	8
3.2 Maduración y formalización (Décadas de 1980–2000)	8
3.3 El cuestionamiento del paradigma (Años 2000–2015)	8
3.4 La reivindicación del monolito (2015–presente)	9
4. Componentes Principales de un Sistema Monolítico	10
4.1 Capa de Presentación (Presentation Layer)	10
4.2 Capa de Lógica de Negocio (Business Logic / Service Layer)	10
4.3 Capa de Acceso a Datos (Data Access / Repository Layer)	10
4.4 Modelos de Dominio (Domain Models)	10
4.5 Módulo de Configuración e Infraestructura	11
4.6 Módulo de Integración Externa	11
5. Flujo de Comunicación Interno	12
5.1 Naturaleza de las invocaciones internas	12
5.2 Flujo de una operación típica	12
5.3 Gestión transaccional	13
5.4 Sincronía versus asincronía	13
6. Ventajas Específicas de la Arquitectura Monolítica	14
6.1 Simplicidad de desarrollo en fases iniciales	14
6.2 Rendimiento superior en operaciones intensivas	14
6.3 Consistencia transaccional sin complejidad distribuida	14
6.4 Depuración y observabilidad simplificadas	15
6.5 Menor infraestructura y costo operacional	15
6.6 Facilidad para pruebas de integración	15
6.7 Refactorización sin barreras de red	15
7. Desventajas y Limitaciones de la Arquitectura Monolítica	17
7.1 Escalabilidad rígida e ineficiente	17
7.2 Acoplamiento tecnológico (Technology Lock-in)	17
7.3 Degradación global ante fallos parciales	17
7.4 Complejidad creciente del código base	18
7.5 Ciclos de despliegue lentos	18
7.6 Dificultad para ciclos de vida independientes por módulo	18
8. Casos de Uso Reales	19
8.1 Stack Overflow	19
8.2 Shopify	19
8.3 Amazon Prime Video (caso de reversión)	19
9. Comparación con Otros Patrones Arquitecturales	21
9.1 Análisis comparativo	22
10. Implementación Práctica	23
10.1 Descripción del sistema implementado	23

10.2 Estructura de carpetas del proyecto	23
10.3 Descripción del código fuente	23
10.3.1 Task.java — Entidad de dominio (Modelo)	24
10.3.2 TaskRepository.java — Capa de acceso a datos	24
10.3.3 TaskService.java — Capa de lógica de negocio	24
10.3.4 TaskMonolith.java — Punto de entrada consola	24
10.3.5 TaskApp.java — Capa de presentación (Interfaz gráfica Swing)	25
10.4 Cómo el código refleja la arquitectura monolítica	25
11. Diagrama Arquitectónico	28
11.1 Descripción del diagrama UML de componentes	28
11.2 Código PlantUML del diagrama	29
12. Tecnologías Asociadas a la Arquitectura Monolítica	30
12.1 Lenguajes de programación	30
12.2 Frameworks de desarrollo	30
12.3 Herramientas de construcción y empaquetado	30
12.4 Sistemas de gestión de bases de datos	30
12.5 Herramientas de despliegue	31
13. Mejores Prácticas para la Arquitectura Monolítica	32
13.1 Modularidad interna explícita	32
13.2 Aplicación de los principios SOLID	32
13.3 Pruebas automatizadas por capas	32
13.4 Gestión cuidadosa de las dependencias externas	32
13.5 Monitoreo y logging estructurado	33
14. Anti-Patrones Comunes en la Arquitectura Monolítica	34
14.1 Big Ball of Mud	34
14.2 Anemic Domain Model	34
14.3 God Class / God Service	34
14.4 Acoplamiento directo a la base de datos	35
14.5 Ignorar la planificación de la extracción de módulos	35
15. Cuándo Usar y Cuándo NO Usar la Arquitectura Monolítica	36
15.1 Cuándo usar arquitectura monolítica	36
15.2 Cuándo NO usar arquitectura monolítica	36
16. Conclusión Técnica	38
17. Referencias	39
18. Anexos	41
Anexo A – Instrucciones de compilación y ejecución	41
Anexo B – Glosario técnico	41

Índice de Figuras y Tablas

Tabla 1. Comparación entre patrones arquitecturales	19
Tabla 2. Criterios para selección de arquitectura monolítica	34
Figura 1. Diagrama UML de componentes – Arquitectura Monolítica	26

1. Introducción

El desarrollo de software moderno opera bajo una paradoja aparente: mientras el discurso técnico contemporáneo se centra en microservicios, arquitecturas orientadas a eventos y patrones cloud-native, la arquitectura monolítica continúa siendo el modelo predominante en el panorama empresarial global. Esta aparente contradicción revela una verdad fundamental sobre la ingeniería de software: la complejidad arquitectónica debe ser siempre proporcional a la complejidad del problema que se pretende resolver.

El presente trabajo de investigación examina en profundidad la Arquitectura Monolítica, el patrón más antiguo, más extendido y, paradójicamente, el más frecuentemente malentendido en los entornos académicos y profesionales contemporáneos. Contrariamente a la percepción popular que lo equipara con código desorganizado o prácticas obsoletas, el monolito bien construido representa una decisión arquitectónica deliberada, coherente y apropiada para una amplia categoría de problemas de software.

A lo largo de este documento se analizan los fundamentos teóricos del patrón, su evolución histórica, sus componentes constitutivos, sus fortalezas y limitaciones intrínsecas, y se presenta una implementación práctica funcional en lenguaje Java que demuestra cómo una aplicación monolítica correctamente diseñada puede ser mantenible, testeable y extensible.

El análisis crítico que se desarrolla en las siguientes secciones tiene por objetivo equipar al estudiante de ingeniería de software con los criterios de decisión necesarios para evaluar cuándo la arquitectura monolítica constituye la elección técnica óptima, y cuándo es pertinente considerar aproximaciones arquitectónicas distribuidas.

2. Definición Formal de la Arquitectura Monolítica

La Arquitectura Monolítica es un patrón de diseño arquitectónico de software en el que todos los componentes funcionales de una aplicación —incluyendo la capa de presentación, la lógica de negocio, el acceso a datos y las integraciones externas— se desarrollan, compilan, despliegan y ejecutan como una única unidad cohesiva e indivisible dentro de un mismo proceso del sistema operativo (Richards, 2015; Fowler & Lewis, 2014).

Esta definición contiene tres elementos conceptuales fundamentales que deben distinguirse con precisión:

- **Unicidad de despliegue (Single Deployment Unit):** toda la aplicación se empaqueta en un único artefacto —un archivo JAR, WAR, EXE o ejecutable equivalente— que se despliega de forma atómica. No existe la posibilidad de actualizar un componente de forma independiente sin redesplegar la totalidad del sistema.
- **Espacio de proceso compartido (Shared Process Space):** todos los módulos comparten el mismo espacio de memoria del proceso anfitrión. Las llamadas entre componentes son invocaciones directas de función en memoria (in-process calls), no comunicaciones de red, lo que elimina la latencia de serialización/deserialización propia de las arquitecturas distribuidas.
- **Código base unificado (Unified Codebase):** el código fuente de todos los módulos reside en un único repositorio o proyecto de compilación. Los cambios en cualquier componente requieren la recompilación y el redespiegue de la totalidad del artefacto.

Es importante distinguir la Arquitectura Monolítica del antipatrón conocido como Big Ball of Mud (Foote & Yoder, 1997), con el que frecuentemente se le confunde. Un monolito bien estructurado puede presentar una organización interna altamente modular, con separación de responsabilidades clara y límites entre capas rigurosamente definidos. La diferencia radica en el modelo de despliegue, no en la calidad interna del código.

Los principios fundamentales sobre los que se basa el patrón son:

1. **Localidad de referencia:** al compartir el mismo espacio de memoria, las operaciones entre módulos son significativamente más rápidas que en sistemas distribuidos.
2. **Consistencia transaccional nativa:** la capacidad de ejecutar transacciones ACID que abarcan múltiples dominios de negocio sin necesidad de protocolos de transacción distribuida.
3. **Simplicidad operacional:** un único artefacto de despliegue reduce drásticamente la complejidad de la infraestructura de orquestación, monitoreo y gestión del ciclo de vida.

3. Historia y Evolución del Patrón

3.1 Orígenes (Décadas de 1950–1980)

La arquitectura monolítica no fue el resultado de una decisión de diseño deliberada, sino la consecuencia natural de las restricciones tecnológicas de los primeros sistemas de cómputo. Durante las décadas de 1950 y 1960, los sistemas de software se desarrollaban sobre hardware mainframe con recursos extremadamente limitados. Los lenguajes de programación de aquella era —FORTRAN (1957), COBOL (1959) y posteriormente C (1972)— compilaban la totalidad del código fuente en un único ejecutable. Los sistemas operativos de tiempo compartido como UNIX reforzaron este modelo al estar diseñados para ejecutar programas como procesos monolíticos con espacios de direcciones diferenciados (Ritchie & Thompson, 1978).

3.2 Maduración y formalización (Décadas de 1980–2000)

Con el advenimiento de los lenguajes orientados a objetos —Smalltalk, C++ y posteriormente Java— el monolito adquirió una estructura interna más sofisticada. La organización en capas (Presentation, Business Logic, Data Access) emergió como práctica estándar durante los años noventa, dando lugar al patrón Three-Tier Architecture que formalizó la separación lógica de responsabilidades dentro de un único proceso de despliegue (Fowler, 2002). Frameworks empresariales como Enterprise JavaBeans (EJB) y Spring Framework (2003) estandarizaron las mejores prácticas para la construcción de aplicaciones monolíticas robustas.

3.3 El cuestionamiento del paradigma (Años 2000–2015)

La democratización de Internet y la necesidad de escalar sistemas a millones de usuarios concurrentes expusieron las limitaciones del monolito en escenarios de alta demanda. Amazon, eBay y Netflix documentaron públicamente sus migraciones hacia arquitecturas de servicios como respuesta a problemas concretos de escalabilidad y velocidad de despliegue (Newman, 2015). El artículo seminal de Fowler y Lewis, "Microservices" (2014), cristalizó el debate y popularizó la idea de que el monolito debía ser reemplazado.

3.4 La reivindicación del monolito (2015–presente)

A partir de 2015, la industria comenzó a documentar los costos reales de la complejidad distribuida. El caso de Amazon Prime Video —que revirtió parte de su arquitectura serverless a un proceso monolítico obteniendo una reducción del 90% en costos operativos (Casado & Bornstein, 2022)— generó un reexamen crítico del paradigma dominante. El concepto de Modular Monolith o Majestic Monolith ganó tracción como alternativa que combina la simplicidad operacional del monolito con los principios de separación de responsabilidades propios de las arquitecturas modulares. Basecamp, GitHub y Shopify son ejemplos contemporáneos de empresas que operan exitosamente con arquitecturas monolíticas a escala (DHH, 2016).

4. Componentes Principales de un Sistema Monolítico

Un sistema monolítico bien diseñado no es un bloque de código indiferenciado. Internamente, se organiza en módulos funcionales con responsabilidades claramente delimitadas que, a pesar de compartir el mismo proceso de ejecución, mantienen interfaces y contratos bien definidos.

4.1 Capa de Presentación (Presentation Layer)

Responsable de gestionar la interacción con el usuario final o con sistemas cliente externos. Contiene los controladores que reciben peticiones, los mecanismos de renderizado de vistas y la lógica de validación de entrada. Esta capa no debe contener lógica de negocio; su única responsabilidad es orquestar la comunicación entre el usuario y los servicios de la aplicación.

4.2 Capa de Lógica de Negocio (Business Logic / Service Layer)

Constituye el núcleo funcional de la aplicación. Contiene las reglas de negocio, los flujos de trabajo y las operaciones que definen el comportamiento específico del dominio. Esta capa es agnóstica respecto a los mecanismos de persistencia y de presentación, operando exclusivamente sobre modelos de dominio. En términos de patrones de diseño, se implementa frecuentemente mediante el patrón Service, complementado con el patrón Domain Model para dominios complejos (Fowler, 2002).

4.3 Capa de Acceso a Datos (Data Access / Repository Layer)

Abstrae los mecanismos de persistencia, proporcionando una interfaz uniforme para las operaciones CRUD independientemente del mecanismo de almacenamiento subyacente. La implementación mediante el patrón Repository permite sustituir el mecanismo de persistencia sin modificar la lógica de negocio.

4.4 Modelos de Dominio (Domain Models)

Las entidades de negocio compartidas transversalmente por todas las capas. En una aplicación de gestión de tareas, entidades como Task, User y Project constituyen el

modelo de dominio. Su diseño determina en gran medida la cohesión y el acoplamiento interno del sistema.

4.5 Módulo de Configuración e Infraestructura

Comprende los mecanismos de inicialización de la aplicación, gestión de dependencias (inyección de dependencias), configuración de logging, gestión de excepciones transversales (cross-cutting concerns) y, en sistemas empresariales, la gestión de transacciones.

4.6 Módulo de Integración Externa

Encapsula la comunicación con sistemas externos —servicios de correo electrónico, APIs de terceros, sistemas de mensajería— aislando al resto de la aplicación de las dependencias externas mediante el patrón Adapter o Anti-Corruption Layer.

5. Flujo de Comunicación Interno

5.1 Naturaleza de las invocaciones internas

El rasgo definitorio del flujo de comunicación en una arquitectura monolítica es que todas las interacciones entre componentes son invocaciones directas de método en el mismo espacio de memoria (in-process method calls). Esta característica distingue fundamentalmente al monolito de cualquier arquitectura distribuida, donde la comunicación entre componentes implica serialización, transmisión de red y deserialización. La ausencia de comunicación de red implica que el flujo de datos opera con latencias del orden de nanosegundos o microsegundos.

5.2 Flujo de una operación típica

El flujo de comunicación en un sistema monolítico de tres capas sigue el siguiente patrón secuencial y síncrono:

4. El cliente emite una solicitud que llega a la Capa de Presentación mediante HTTP/HTTPS.
5. La Capa de Presentación valida la solicitud y delega la ejecución a los servicios de la Capa de Lógica de Negocio mediante una invocación directa de método.
6. La Capa de Lógica de Negocio ejecuta las reglas de dominio e interactúa con repositorios de la Capa de Acceso a Datos cuando requiere persistir o recuperar información.
7. La Capa de Acceso a Datos traduce las operaciones a consultas sobre el mecanismo de persistencia y retorna objetos de dominio.
8. El resultado asciende en cascada hasta la Capa de Presentación, que serializa la respuesta y la transmite al cliente.

5.3 Gestión transaccional

Una ventaja significativa del modelo de comunicación monolítico es la simplicidad de la gestión transaccional. Dado que todas las operaciones ocurren dentro del mismo proceso, las transacciones ACID pueden abarcar múltiples operaciones de dominio sin requerir mecanismos de coordinación distribuida. En el ecosistema Java, la anotación `@Transactional` de Spring gestiona transparentemente el ciclo de vida transaccional, garantizando atomicidad, consistencia, aislamiento y durabilidad de forma trivial.

5.4 Sincronía versus asincronía

Si bien el modelo de comunicación por defecto es síncrono, es posible incorporar mecanismos de procesamiento asíncrono sin abandonar el paradigma monolítico. El uso de colas en memoria (`BlockingQueue` de Java), hilos de procesamiento en background o frameworks reactivos permite desacoplar temporalmente productor y consumidor de eventos sin introducir la complejidad operacional de brokers de mensajería externos.

6. Ventajas Específicas de la Arquitectura Monolítica

6.1 Simplicidad de desarrollo en fases iniciales

En las etapas tempranas de un proyecto, la arquitectura monolítica elimina la fricción propia de los sistemas distribuidos. El equipo puede centrarse en la implementación de funcionalidades de negocio sin necesidad de definir contratos de servicio, gestionar consistencia eventual ni configurar infraestructura de orquestación. Esta reducción de la carga cognitiva se traduce en mayor velocidad de iteración, lo cual es crítico cuando la validación del producto con usuarios reales es la prioridad. Newman (2015) señala que comenzar con un monolito permite comprender los límites de los dominios de negocio antes de intentar distribuirlos.

6.2 Rendimiento superior en operaciones intensivas

Las invocaciones en memoria entre componentes de un monolito son entre dos y cuatro órdenes de magnitud más rápidas que las llamadas de red equivalentes en un sistema distribuido. En aplicaciones con flujos de trabajo que requieren numerosas interacciones entre diferentes dominios de negocio, este diferencial de rendimiento puede ser determinante. Un flujo que en un monolito implica diez invocaciones de método —con latencia total de microsegundos— podría implicar diez llamadas HTTP en un sistema de microservicios, acumulando decenas de milisegundos y aumentando la probabilidad de fallos en cascada.

6.3 Consistencia transaccional sin complejidad distribuida

La gestión de transacciones ACID en una arquitectura monolítica es trivialmente simple. La atomicidad entre operaciones que abarcan múltiples entidades de dominio se garantiza mediante los mecanismos nativos de los sistemas gestores de bases de datos relacionales. En contraste, implementar consistencia transaccional en microservicios requiere patrones complejos como Saga o el protocolo de compromiso en dos fases (2PC), que introducen complejidad accidental significativa y son fuentes habituales de defectos difíciles de reproducir (Richardson, 2018).

6.4 Depuración y observabilidad simplificadas

El diagnóstico de problemas en un sistema monolítico es inherentemente más sencillo. La traza de ejecución de una operación está contenida en un único proceso, accesible mediante un único stream de logs, un único agente de profiling y un único depurador interactivo. En contraste, la depuración de microservicios requiere herramientas de trazabilidad distribuida (Jaeger, Zipkin) para correlacionar eventos que se propagan a través de múltiples servicios, cada uno con sus propios logs y métricas.

6.5 Menor infraestructura y costo operacional

El despliegue de un sistema monolítico requiere una infraestructura significativamente menos compleja. Un monolito puede ejecutarse en un único servidor o en un pequeño cluster de nodos balanceados. La orquestación de decenas de microservicios independientes requiere plataformas como Kubernetes, service discovery, gateways de API y múltiples bases de datos. El equipo de Amazon Prime Video documentó una reducción del 90% en costos de infraestructura tras consolidar parte de su arquitectura distribuida en un proceso monolítico (Casado & Bornstein, 2022).

6.6 Facilidad para pruebas de integración

Probar el comportamiento completo del sistema es considerablemente más sencillo cuando todos sus componentes están en el mismo proceso. Las pruebas de integración pueden ejecutarse sin infraestructura adicional de simulación de servicios externos, reduciendo la complejidad del entorno de pruebas y aumentando la confiabilidad de los resultados.

6.7 Refactorización sin barreras de red

Cuando los límites de los módulos de negocio cambian —situación habitual en proyectos en fase de maduración— modificar las interfaces entre componentes en un monolito es una operación local que el compilador puede verificar estáticamente. En un sistema de microservicios, redefinir el contrato de un servicio implica

actualizaciones coordinadas de múltiples equipos, versionado de APIs y períodos de deprecación extendidos.

7. Desventajas y Limitaciones de la Arquitectura Monolítica

7.1 Escalabilidad rígida e ineficiente

La limitación más citada de la arquitectura monolítica es su modelo de escalabilidad. Dado que todos los componentes se empaquetan en un único artefacto, la escala horizontal implica replicar la totalidad de la aplicación, incluyendo módulos cuya demanda no justifica la replicación. Si el módulo de recomendaciones es el cuello de botella, es necesario desplegar instancias adicionales del sistema completo solo para escalar esa capacidad. Este desperdicio de recursos es aceptable a escalas moderadas, pero se vuelve prohibitivamente costoso en sistemas de alta demanda con patrones de carga heterogéneos.

7.2 Acoplamiento tecnológico (Technology Lock-in)

Un monolito se construye utilizando un único stack tecnológico: un lenguaje de programación, un framework, una versión del runtime. La introducción de nuevas tecnologías o la actualización de dependencias críticas afecta a la totalidad del sistema. Si una librería central es vulnerable y requiere actualizarse, la actualización puede tener efectos en cascada sobre componentes no relacionados. Esta rigidez tecnológica puede convertirse en deuda técnica progresiva cuando el ecosistema de herramientas evoluciona.

7.3 Degradación global ante fallos parciales

En un sistema monolítico, un módulo con una fuga de memoria, un hilo bloqueado o una consulta ineficiente puede degradar el rendimiento del sistema completo o provocar su caída total. La ausencia de aislamiento de fallos entre componentes significa que una funcionalidad de baja criticidad puede comprometer la disponibilidad de funcionalidades críticas. Las arquitecturas distribuidas pueden implementar mecanismos de degradación elegante que aíslan los fallos a servicios específicos.

7.4 Complejidad creciente del código base

A medida que un sistema monolítico crece en funcionalidades, el código base tiende a volverse progresivamente más difícil de comprender y modificar. Sin disciplina arquitectónica estricta, los módulos comienzan a acoplarse en formas no previstas, las dependencias circulares emergen y la comprensión del impacto de un cambio requiere el análisis de una porción cada vez mayor del sistema. Este fenómeno —denominado Big Ball of Mud (Foote & Yoder, 1997)— no es inherente al patrón, pero es una degeneración frecuente sin una modularidad explícita.

7.5 Ciclos de despliegue lentos

La necesidad de recompilar, reempaquetar y redespargar el sistema completo para incorporar cualquier cambio —incluso uno menor en un módulo periférico— implica ciclos de entrega potencialmente lentos. En organizaciones con múltiples equipos trabajando en el mismo código base, la coordinación de los despliegues puede introducir cuellos de botella organizacionales. Fowler describe este fenómeno como un inhibidor de la práctica de Continuous Deployment en monolitos grandes (Fowler & Lewis, 2014).

7.6 Dificultad para ciclos de vida independientes por módulo

En entornos donde diferentes módulos funcionales tienen ciclos de vida distintos —un módulo de procesamiento de pedidos con alta disponibilidad versus un módulo de reportes que puede tolerar mantenimientos— la arquitectura monolítica no permite gestionarlos de forma independiente. Cualquier ventana de mantenimiento afecta a la totalidad del sistema.

8. Casos de Uso Reales

8.1 Stack Overflow

Stack Overflow, con más de 100 millones de visitantes únicos mensuales, ha operado históricamente sobre una arquitectura monolítica en ASP.NET. El sistema procesa millones de solicitudes diarias desde un pequeño cluster de servidores físicos, demostrando que la escalabilidad mediante optimización vertical y caching agresivo puede ser una alternativa viable a la escalabilidad horizontal mediante distribución. El equipo de infraestructura ha documentado que su arquitectura monolítica bien optimizada, ejecutándose sobre hardware de alto rendimiento, les permite operar con una fracción de la infraestructura que requeriría un sistema equivalente de microservicios, manteniendo una organización de ingeniería reducida con alta eficiencia operacional (Atwood & Spolsky, 2008).

8.2 Shopify

Shopify, plataforma de comercio electrónico que procesa millones de transacciones diarias, fue construida como un monolito en Ruby on Rails y permaneció como tal durante años de crecimiento acelerado. Incluso cuando la empresa comenzó a extraer ciertos componentes como servicios independientes, mantuvo el núcleo de la plataforma como un monolito modular. El caso de Shopify ilustra el principio de Strangler Fig Pattern (Fowler, 2004): la migración gradual de un monolito hacia servicios específicos cuando la presión de escala lo justifica, sin intentar una reescritura total. La mayor parte de las funcionalidades del núcleo de comercio continúan funcionando como un monolito modular en producción (Shopify Engineering Blog, 2019).

8.3 Amazon Prime Video (caso de reversión)

El caso más documentado de reversión hacia arquitectura monolítica en la industria reciente es el del equipo de monitoreo de calidad de video de Amazon Prime Video. En 2023, el equipo publicó un artículo técnico describiendo cómo su sistema de monitoreo, originalmente construido sobre una arquitectura serverless distribuida, fue migrado a un proceso monolítico, obteniendo como resultado una reducción del 90%

en costos de infraestructura y una mejora significativa en el rendimiento (Casado & Bornstein, 2022). La causa raíz era que la arquitectura distribuida imponía un costo de comunicación desproporcionado respecto a la complejidad del problema que resolvía. Este caso es frecuentemente citado como evidencia de que la adopción de arquitecturas distribuidas debe estar justificada por necesidades concretas de escala, no por tendencias tecnológicas.

9. Comparación con Otros Patrones Arquitecturales

La siguiente tabla comparativa analiza las características fundamentales de la Arquitectura Monolítica en contraste con los otros tres patrones estudiados en esta asignación.

Criterio	Monolítico	Cliente-Servidor	MVC	En Capas
Unidad de despliegue	Único artefacto (JAR/WAR)	Servidor + clientes separados	Único (incluye M-V-C)	Múltiples tiers (n-tier)
Comunicación interna	In-process (RAM)	Red (HTTP, TCP/IP)	In-process por patrón	In-process o red por tier
Escalabilidad horizontal	Limitada (réplica total)	Alta (solo servidor)	Limitada	Moderada (por capa)
Complejidad inicial	Baja	Media	Baja – Media	Media – Alta
Consistencia transaccional	Alta (ACID nativa)	Depende del servidor	Alta (ACID nativa)	Alta (ACID nativa)
Independencia de módulos	Baja	Alta (C-S independientes)	Media (por roles MVC)	Media (por capas)
Velocidad de despliegue	Un único despliegue	Servidor + clientes por separado	Un único despliegue	Puede ser por capas
Latencia entre componentes	Nanosegundos (RAM)	Milisegundos (red)	Nanosegundos (RAM)	Nanosegundos – ms
Caso de uso ideal	Startups, MVPs, apps medianas	Múltiples clientes/dispositivos	UI compleja, apps web	Sistemas empresariales grandes
Dificultad de testing	Baja (todo en un proceso)	Media (mocks de red)	Media (separación MVC)	Media (múltiples capas)

Tabla 1. Comparación entre patrones arquitecturales. Elaboración propia basada en Richards (2015) y Fowler (2002).

9.1 Análisis comparativo

La comparación evidencia que ninguno de los cuatro patrones es universalmente superior. El patrón monolítico destaca en dos dimensiones críticas para proyectos en fases iniciales: la baja complejidad de entrada y la consistencia transaccional nativa. La Arquitectura en Capas es funcionalmente similar al monolito en su modelo de despliegue, pero introduce mayor formalidad en la separación de responsabilidades. El patrón MVC introduce un eje de separación adicional —entre Model, View y Controller— especialmente apropiado para interfaces de usuario complejas, y puede coexistir perfectamente dentro de un monolito. La arquitectura Cliente-Servidor se diferencia por la separación física entre clientes y servidor, ofreciendo mayor independencia pero añadiendo complejidad de comunicación.

10. Implementación Práctica

10.1 Descripción del sistema implementado

Se implementó un Sistema de Gestión de Tareas (Task Management System) en Java, diseñado como un monolito modular. El sistema permite crear, consultar, actualizar y eliminar tareas, así como filtrarlas por estado y prioridad. La implementación demuestra los principios fundamentales de la arquitectura monolítica: unidad de despliegue, espacio de proceso compartido y código base unificado.

El sistema se organiza internamente en paquetes (packages) de Java que emulan la separación de capas característica de un monolito bien estructurado: model, repository, service y main (presentación/orquestación). A pesar de esta separación lógica, toda la aplicación compila y se ejecuta como un único artefacto JAR.

10.2 Estructura de carpetas del proyecto

La estructura de directorios refleja la organización modular interna del monolito y cumple con la convención de separación por responsabilidades:

```
task-monolith/  
├── src/  
│   ├── main/  
│   │   └── java/  
│   │       └── com/taskmonolith/  
│   │           ├── model/           ← Entidades de dominio (Task.java)  
│   │           ├── repository/      ← Acceso a datos (TaskRepository.java)  
│   │           ├── service/         ← Lógica de negocio (TaskService.java)  
│   │           ├── TaskMonolith.java ← Punto de entrada (versión consola)  
│   │           └── TaskApp.java      ← Interfaz gráfica (Swing)  
├── pom.xml                         ← Configuración Maven: single deployment  
├── unit  
└── README.md                       ← Instrucciones de instalación y ejecución
```

10.3 Descripción del código fuente

El sistema está compuesto por cinco archivos Java organizados en paquetes que representan cada capa de la arquitectura. El código fuente completo, comentado y funcional, está disponible en el repositorio público del grupo:

<https://github.com/HumanoidCat/task-monolith>

10.3.1 Task.java — Entidad de dominio (Modelo)

Representa la entidad central del dominio. Es un POJO (Plain Old Java Object) que encapsula todos los datos de una tarea: identificador único generado automáticamente con `UUID`, título, descripción, estado (`PENDING`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`), prioridad (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`), responsable y marcas de tiempo de creación y actualización. Define getters y setters con validaciones: el título no puede estar vacío, la prioridad no puede ser nula. No depende de ninguna otra clase del proyecto, lo que lo hace reutilizable en cualquier capa.

10.3.2 TaskRepository.java — Capa de acceso a datos

Abstrae el mecanismo de persistencia. En esta prueba de concepto utiliza un `HashMap<String, Task>` en memoria que simula una base de datos. Expone operaciones CRUD: `save()`, `findById()`, `findAll()`, `findByStatus()`, `findByPriority()` y `delete()`. Utiliza la API de Streams de Java para los filtros. La capa de servicio no conoce cómo se almacenan los datos internamente, lo que permite reemplazar el `HashMap` por una base de datos real sin modificar ninguna otra capa.

10.3.3 TaskService.java — Capa de lógica de negocio

Núcleo del sistema. Recibe un `TaskRepository` por constructor (inyección de dependencias, principio D de SOLID) y concentra todas las reglas de negocio del sistema. Implementa las tres reglas fundamentales: una tarea `CANCELLED` es inmutable, una tarea `COMPLETED` no puede retroceder de estado, y solo las tareas `COMPLETED` pueden eliminarse. Expone métodos como `createTask()`, `updateStatus()`, `deleteTask()` y `getAllTasks()`. Ninguna otra capa replica estas validaciones.

10.3.4 TaskMonolith.java — Punto de entrada consola

Contiene el método `main()` de la versión de consola. Instancia directamente un `TaskRepository` y un `TaskService` y ejecuta una demo en cinco fases: crear tareas, cambiar estados, aplicar reglas de negocio, filtrar por estado y eliminar. Demuestra que todas las capas operan dentro del mismo proceso JVM sin ninguna comunicación de red.

10.3.5 TaskApp.java — Capa de presentación (Interfaz gráfica Swing)

Implementa la interfaz gráfica del sistema usando Java Swing, incluido en el JDK sin dependencias externas. Extiende `JFrame` e instancia las mismas capas internas (`TaskRepository` y `TaskService`) que usa la versión de consola. Esto demuestra un principio clave del monolito: la lógica de negocio es completamente independiente de la interfaz. La ventana permite crear tareas con formulario, listarlas, cambiar su estado con botones de colores y eliminarlas. Es el punto de entrada del JAR ejecutable: `java -jar task-monolith-1.0.jar` abre esta ventana directamente.

10.4 Cómo el código refleja la arquitectura monolítica

El código presentado demuestra los tres principios fundamentales de la arquitectura monolítica de forma explícita y verificable. A continuación se analiza paso a paso cómo cada decisión de implementación refleja un principio arquitectónico concreto.

Paso 1 — Un solo artefacto de despliegue (Single Deployment Unit)

El archivo `pom.xml` configura Maven para producir un único artefacto ejecutable: `task-monolith.jar`. Al ejecutar `mvn clean package`, los cinco archivos fuente (`Task.java`, `TaskRepository.java`, `TaskService.java`, `TaskMonolith.java` y `TaskApp.java`) se compilan juntos y quedan empaquetados en ese único archivo. No existen microservicios separados, contenedores Docker independientes ni APIs intermedias. Todo el sistema arranca con un único comando: `java -jar task-monolith.jar`. Eso es exactamente la definición operacional del Single Deployment Unit.

Paso 2 — Comunicación entre capas en memoria (Shared Process Space)

Cada vez que `TaskService` llama a `repository.save(task)` o `repository.findById(id)`, esa invocación es una llamada de método Java pura: ocurre dentro de la misma JVM, en el mismo espacio de memoria del proceso, en nanosegundos. No hay serialización a JSON, no hay socket de red, no hay código de manejo de errores de red ni timeouts. En contraste, si el mismo sistema estuviera implementado como microservicios, esa misma línea requeriría una llamada HTTP, serialización de objetos, manejo de códigos de respuesta (200, 404, 500) y una

latencia de entre 5 y 50 ms. El código hace visible la diferencia: una sola línea vs. un cliente HTTP completo.

Paso 3 — Separación de capas interna (Unified Codebase modular)

Aunque todo el código vive en un solo artefacto, la estructura de paquetes impone fronteras claras entre capas: `com.taskmonolith.model` (dominio), `com.taskmonolith.repository` (persistencia), `com.taskmonolith.service` (negocio) y el paquete raíz (presentación). Esta separación es verificable: `TaskApp.java` y `TaskMonolith.java` nunca importan clases de `repository` directamente; toda la interacción pasa siempre por `TaskService`. La presentación no conoce cómo se almacenan los datos: eso es separación de capas dentro de un monolito.

Paso 4 — Reglas de negocio en un único lugar

Las tres reglas de dominio del sistema (una tarea cancelada es inmutable, una tarea completada no puede retroceder de estado, y solo las tareas completadas pueden eliminarse) están implementadas exclusivamente en `TaskService.java`, métodos `updateStatus()` y `deleteTask()`. Ni `TaskApp.java` ni `TaskRepository.java` replican esas validaciones. Esto es el principio S de SOLID (Single Responsibility) aplicado a nivel arquitectónico: hay un único punto de verdad para las reglas de negocio. Cualquier modificación a esas reglas requiere cambiar exactamente un archivo.

Paso 5 — Inyección de dependencias (Constructor Injection — principio D de SOLID)

El constructor de `TaskService` recibe un `TaskRepository` como parámetro en lugar de crearlo internamente. Esta decisión tiene dos consecuencias arquitectónicas directas: primero, `TaskService` no tiene acoplamiento concreto con la implementación de persistencia (podría recibir un mock en los tests); segundo, la dependencia es explícita en la firma del constructor, lo que hace la arquitectura legible directamente desde el código sin necesidad de documentación adicional. Este patrón (Constructor Injection) es la forma más limpia de aplicar el principio de Inversión de Dependencias en un monolito sin frameworks externos.

Paso 6 — Dos puntos de entrada, mismas capas internas

El sistema tiene dos puntos de entrada: `TaskMonolith.java` (consola) y `TaskApp.java` (interfaz gráfica Swing). Ambos instancian el mismo `TaskRepository` y el mismo `TaskService`. Esto demuestra un principio central del monolito bien diseñado: la lógica de negocio es completamente independiente de la interfaz que la consume. Cambiar de una UI de consola a una GUI gráfica no requirió modificar ni una línea del servicio ni del repositorio. Esa independencia es la base para poder añadir en el futuro una tercera interfaz (por ejemplo, una API REST) reutilizando exactamente el mismo núcleo.

- **Unidad de despliegue única:** los cuatro archivos compilados mediante mvn package generan un único archivo `task-monolith.jar`. No existe ningún servicio externo ni comunicación de red entre los módulos.
- **Invocaciones en memoria:** la línea `repository.save(task)` en `TaskService.java` es una llamada directa de método Java, sin serialización ni HTTP. La latencia es de nanosegundos, lo que contrasta directamente con un microservicio equivalente que requeriría una llamada REST o gRPC.
- **Código base unificado:** los modelos de dominio (`Task`, `Task.Status`, `Task.Priority`) son clases Java compartidas directamente entre todas las capas sin DTOs de traducción, contratos de API ni esquemas de serialización intermedios.
- **Reglas de negocio centralizadas:** toda la lógica de dominio reside exclusivamente en `TaskService.java`: la validación de que solo pueden eliminarse tareas completadas, y que las tareas canceladas no pueden modificarse, están en un único lugar del código base, facilitando su mantenimiento y auditoría.

11.2 Código PlantUML del diagrama

El siguiente código PlantUML puede pegarse directamente en <https://www.plantuml.com/plantuml/uml/> para generar el diagrama:

```
@startuml
title Sistema de Gestión de Tareas - Arquitectura Monolítica

package "task-monolith.jar (Single Deployment Unit)" #DDEEFF {
    component [Main\n(Presentation Layer)] as Main
    component [TaskService\n(Business Logic Layer)] as Service
    component [TaskRepository\n(Data Access Layer)] as Repo
    component [Task\n(Domain Model)] as Model
    database [In-Memory Store\n(HashMap)] as Store
}

[Console / Client] --> Main : uses
Main ..> Service : calls (in-process)
Service ..> Repo : calls (in-process)
Service ..> Model : uses
Repo ..> Model : reads/writes
Repo ..> Store : persists

note right of Main
    Single Entry Point
    java -jar task-monolith.jar
end note

@enduml
```

12. Tecnologías Asociadas a la Arquitectura Monolítica

12.1 Lenguajes de programación

La arquitectura monolítica es compatible con cualquier lenguaje de propósito general. Los más utilizados en entornos de producción son Java con el ecosistema Spring Boot para aplicaciones empresariales, Ruby con Ruby on Rails adoptado por GitHub y Shopify, Python con Django o Flask, C# con ASP.NET en el ecosistema Microsoft, y Go para aplicaciones de rendimiento crítico.

12.2 Frameworks de desarrollo

Spring Boot (Java/Kotlin) es el framework más extendido para el desarrollo de monolitos empresariales modernos. Provee gestión de dependencias por inyección, configuración declarativa, gestión de transacciones e integración con múltiples motores de persistencia. El modelo de despliegue de Spring Boot empaqueta toda la aplicación —incluyendo el servidor web embebido— en un único archivo JAR ejecutable, simplificando radicalmente el proceso de despliegue y evidenciando el principio de unidad de despliegue.

12.3 Herramientas de construcción y empaquetado

Apache Maven y Gradle son las herramientas estándar de construcción en el ecosistema Java para producir el artefacto monolítico final. Maven, mediante su ciclo de vida de construcción, compila, ejecuta pruebas, empaqueta y despliega el sistema completo con el comando `mvn clean package`. Esta unicidad del proceso de construcción es un reflejo directo del modelo de despliegue monolítico.

12.4 Sistemas de gestión de bases de datos

Los sistemas monolíticos suelen operar con una única base de datos relacional: PostgreSQL, MySQL, Oracle Database o Microsoft SQL Server son las opciones predominantes en entornos empresariales. Los ORM como Hibernate (Java) o ActiveRecord (Rails) simplifican la interacción con la base de datos abstrauyendo las operaciones SQL mediante objetos del modelo de dominio.

12.5 Herramientas de despliegue

El despliegue de un monolito puede realizarse mediante herramientas convencionales como scripts de shell, plataformas PaaS (Heroku, Render, Fly.io) o mediante contenedores Docker con un único Dockerfile que empaqueta el artefacto JAR. Este último enfoque combina la simplicidad del monolito con la portabilidad y reproducibilidad de los contenedores.

13. Mejores Prácticas para la Arquitectura Monolítica

13.1 Modularidad interna explícita

La práctica más crítica para preservar la mantenibilidad de un monolito es la definición y cumplimiento de módulos internos con fronteras explícitas. Cada módulo debe exponer únicamente una API pública mínima y mantener sus detalles de implementación encapsulados. En Java, esto puede implementarse mediante el Java Platform Module System (JPMS, Java 9+), que permite declarar dependencias entre módulos de forma explícita y verificada por el compilador.

13.2 Aplicación de los principios SOLID

Los principios SOLID son especialmente relevantes en el contexto monolítico, donde la ausencia de fronteras de red puede facilitar la proliferación de dependencias no intencionales entre componentes. El principio de Inversión de Dependencias es la base del patrón Repository utilizado en la implementación presentada: el servicio depende de una abstracción, no de una implementación concreta, lo que facilita las pruebas unitarias y la sustitución del mecanismo de persistencia.

13.3 Pruebas automatizadas por capas

La pirámide de pruebas —pruebas unitarias en la base, pruebas de integración en el nivel intermedio y pruebas end-to-end en el vértice— es especialmente apropiada para sistemas monolíticos. Las pruebas unitarias pueden verificar la lógica de negocio en TaskService de forma aislada utilizando mocks del repositorio (Mockito en Java); las pruebas de integración pueden validar el comportamiento completo del sistema cargando todos los componentes en el mismo proceso de prueba.

13.4 Gestión cuidadosa de las dependencias externas

Dado que todas las dependencias de un monolito residen en el mismo proceso, la gestión de versiones debe realizarse con especial cuidado para evitar conflictos de classpath. El uso de herramientas de gestión de dependencias con resolución de

versiones determinista (Maven con BOM —Bill of Materials— o Gradle con constraint resolution) mitiga el riesgo de incompatibilidades entre librerías.

13.5 Monitoreo y logging estructurado

Implementar logging estructurado en formato JSON (legible por herramientas de análisis como ELK Stack o Grafana Loki) desde las primeras etapas del desarrollo facilita el diagnóstico en producción. Herramientas como Micrometer permiten exponer métricas de rendimiento del sistema —tiempo de respuesta, uso de memoria, throughput— con configuración mínima.

14. Anti-Patrones Comunes en la Arquitectura Monolítica

14.1 Big Ball of Mud

El antipatrón más frecuente y destructivo en sistemas monolíticos es el Big Ball of Mud (Foote & Yoder, 1997): un sistema sin estructura arquitectónica reconocible, donde la lógica de negocio se mezcla con el código de acceso a datos y la presentación, las dependencias entre módulos son arbitrarias y bidireccionales, y ningún desarrollador puede predecir el impacto de un cambio. La prevención requiere disciplina arquitectónica sostenida: revisiones de código que verifiquen el cumplimiento de las fronteras de capa y herramientas de análisis estático de dependencias como ArchUnit para Java.

14.2 Anemic Domain Model

El Modelo de Dominio Anémico (Fowler, 2003) es un antipatrón donde las entidades de dominio son simples contenedores de datos sin comportamiento propio, mientras que la lógica de negocio se concentra en servicios que manipulan esas entidades de forma procedimental. Aunque prevalente, viola el principio de encapsulamiento orientado a objetos. El antídoto es el Domain Model rico, donde las entidades contienen el comportamiento que les corresponde.

14.3 God Class / God Service

La concentración de responsabilidades excesivas en una única clase o servicio —denominada God Class— viola el principio de Responsabilidad Única. En sistemas monolíticos, es frecuente encontrar servicios que crecen hasta contener cientos de métodos no relacionados. La solución es la descomposición vertical por dominio de negocio: en lugar de un único UserService, se definen servicios separados como AuthService, ProfileService y NotificationService con responsabilidades claramente delimitadas.

14.4 Acoplamiento directo a la base de datos

El uso directo de consultas SQL nativas en la capa de lógica de negocio crea un acoplamiento fuerte que hace que cualquier cambio en el esquema de base de datos requiera modificaciones en múltiples capas. El patrón Repository provee la abstracción necesaria para aislar estas capas, como se demuestra en la implementación presentada.

14.5 Ignorar la planificación de la extracción de módulos

Un antipatrón estratégico es asumir que el monolito permanecerá como la arquitectura definitiva sin considerar los escenarios en los que la extracción de módulos a servicios independientes podría ser beneficiosa. La ausencia de fronteras de módulo bien definidas convierte la eventual migración en una operación prohibitivamente costosa. La mejor práctica es construir el monolito con módulos internos que respeten los límites de los subdominios de negocio (Bounded Contexts del Domain-Driven Design).

15. Cuándo Usar y Cuándo NO Usar la Arquitectura Monolítica

15.1 Cuándo usar arquitectura monolítica

Escenario	Justificación técnica
Proyecto nuevo / MVP	El dominio aún no está bien comprendido. Un monolito permite iterar rápidamente sin el costo de definir contratos de API entre servicios.
Equipo pequeño (2–8 ingenieros)	La coordinación de múltiples servicios independientes requiere overhead operacional que un equipo pequeño no puede absorber eficientemente.
Dominio de negocio acotado	Cuando los casos de uso están claramente definidos y la complejidad del dominio es manejable, el monolito provee la solución más eficiente.
Presupuesto de infraestructura limitado	La operación de microservicios requiere inversión en orquestación, service mesh y bases de datos por servicio. El monolito reduce drásticamente estos costos.
Consistencia transaccional crítica	Sistemas financieros donde la atomicidad entre múltiples entidades de dominio es no negociable se benefician del soporte nativo de transacciones ACID.
Patrones de carga uniformes	Cuando todos los módulos experimentan patrones de demanda similares, la escalabilidad granular de los microservicios no aporta valor.

Tabla 2. Criterios para la selección de arquitectura monolítica. Elaboración propia basada en Newman (2015) y Richards (2015).

15.2 Cuándo NO usar arquitectura monolítica

- **Escala masiva con cargas heterogéneas:** cuando distintos módulos experimentan diferencias de demanda de varios órdenes de magnitud, la replicación total del monolito para escalar un único módulo es ineficiente.
- **Múltiples equipos con autonomía de despliegue:** en organizaciones con 10 o más equipos trabajando sobre el mismo sistema, la coordinación de despliegues se convierte en un cuello de botella organizacional.

- **Alta disponibilidad diferenciada por módulo:** cuando componentes distintos requieren SLAs de disponibilidad diferentes y estrategias de mantenimiento independientes.
- **Necesidad de diversidad tecnológica:** cuando diferentes subdominios se benefician de tecnologías específicas (Python para ML, Go para streaming, Java para reporting), el monolito impone una uniformidad tecnológica limitante.
- **Velocidades de evolución muy diferentes:** cuando un módulo requiere múltiples despliegues diarios mientras otros son altamente estables, el monolito obliga a un ritmo único de despliegue no óptimo para ninguno.

16. Conclusión Técnica

El análisis realizado a lo largo de este trabajo permite extraer conclusiones que van más allá de la descripción superficial del patrón. La arquitectura monolítica es, ante todo, una herramienta cuya idoneidad depende enteramente del contexto en el que se aplica.

La evidencia empírica reciente, ejemplificada por el caso de Amazon Prime Video y la persistencia de monolitos de alto rendimiento como Stack Overflow y Shopify, desafía la narrativa simplista que equipara microservicios con modernidad y monolito con deuda técnica. La complejidad distribuida tiene un costo real —operacional, cognitivo y económico— que solo se justifica cuando los beneficios que aporta superan concretamente ese costo.

La implementación práctica desarrollada en este trabajo demuestra que un monolito bien diseñado puede ser modular, testeable, mantenible y extensible. La separación de responsabilidades entre las capas Model, Repository, Service y Main no requiere fronteras de red para ser efectiva; requiere disciplina de diseño y adherencia a principios de ingeniería de software probados.

El principio más importante que emerge de este estudio es el de la evolución incremental: comenzar con un monolito modular, comprender profundamente los subdominios de negocio, identificar con evidencia empírica los módulos que se beneficiarían de la extracción como servicios independientes, y ejecutar esa extracción de forma gradual. Esta estrategia —denominada Monolith First por Martin Fowler— minimiza el riesgo de invertir en la complejidad de una arquitectura distribuida prematura para un dominio que aún no la justifica.

En síntesis, dominar la arquitectura monolítica no es un requisito del pasado: es una competencia fundamental del arquitecto de software contemporáneo que debe evaluar con criterio cuándo la simplicidad es la solución correcta y cuándo la complejidad adicional de la distribución está justificada por necesidades de negocio concretas.

17. Referencias

Atwood, J., & Spolsky, J. (2008). Stack Overflow architecture. Coding Horror Blog. <https://stackoverflow.blog/2008/09/stack-overflow-architecture>

Casado, M., & Bornstein, M. (2022, May 4). Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%. Amazon Prime Video Tech Blog. <https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90>

DHH (David Heinemeier Hansson). (2016, March 4). The majestic monolith. Signal v. Noise. <https://m.signalvnoise.com/the-majestic-monolith/>

Foot, B., & Yoder, J. (1997). Big ball of mud. In Proceedings of the 4th Conference on Pattern Languages of Programs (PLoP). University of Illinois.

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley Professional.

Fowler, M. (2003). Anemic domain model. Martin Fowler's Bliki. <https://martinfowler.com/bliki/AnemicDomainModel.html>

Fowler, M. (2004). Strangler fig application. Martin Fowler's Bliki. <https://martinfowler.com/bliki/StranglerFigApplication.html>

Fowler, M., & Lewis, J. (2014, March 25). Microservices. Martin Fowler's Bliki. <https://martinfowler.com/articles/microservices.html>

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Newman, S. (2015). Building microservices: Designing fine-grained systems. O'Reilly Media.

Richards, M. (2015). Software architecture patterns. O'Reilly Media.

Richardson, C. (2018). Microservices patterns: With examples in Java. Manning Publications.

Ritchie, D. M., & Thompson, K. (1978). The UNIX time-sharing system. *Bell System Technical Journal*, 57(6), 1905–1929.
<https://doi.org/10.1002/j.1538-7305.1978.tb02136.x>

Shopify Engineering Blog. (2019). Shopify's architecture to handle flash sales.
<https://shopify.engineering/surviving-flashes-of-high-write-traffic-using-queues>

18. Anexos

Anexo A – Instrucciones de compilación y ejecución

El código completo del sistema se encuentra en el repositorio público de GitHub del grupo. Las instrucciones de ejecución son las siguientes:

1. Clonar el repositorio: `git clone https://github.com/HumanoidCat/task-monolith.git`
2. Navegar al directorio del proyecto: `cd task-monolith`
3. Compilar con Maven (requiere JDK 17+): `mvn clean package`
4. Ejecutar el artefacto monolítico único: `java -jar target/task-monolith-1.0.jar`

Anexo B – Glosario técnico

- **Single Deployment Unit:** artefacto de software que empaqueta la totalidad de los componentes del sistema en un único elemento desplegable (JAR, WAR, EXE).
- **In-Process Call:** invocación directa de método entre objetos que comparten el mismo espacio de memoria del proceso, sin comunicación de red.
- **ACID:** acrónimo de las propiedades de transacciones: Atomicidad, Consistencia, Isolamiento (Isolation) y Durabilidad.
- **MVP (Minimum Viable Product):** versión mínima de un producto con las funcionalidades necesarias para validar una hipótesis de negocio con usuarios reales.
- **PoC (Proof of Concept):** implementación funcional mínima destinada a demostrar la viabilidad técnica de un concepto arquitectónico.
- **Bounded Context:** término del Domain-Driven Design que denota un límite explícito dentro del cual un modelo de dominio específico es válido y consistente.

- **Strangler Fig Pattern:** estrategia de migración gradual de un sistema legado, extrayendo funcionalidades de forma incremental sin reescribir el sistema completo.
- **SOLID:** acrónimo de cinco principios de diseño orientado a objetos: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation y Dependency Inversion.